

DANMARKS TEKNISKE UNIVERSITET



MODELING CO₂ CONCENTRATIONS USING HIDDEN
MARKOV MODELS

2026

SPECIAL COURSE IN STATISTICAL MODELLING

Mads Holme Håkonsson
s234936

Supervisor
Jan Klobbenborg Møller

30. maj 2026

AI Declaration

AI tools has been used for formatting LaTeX figures, tables and equations and grammar checking in the report. No AI has been used for for writing the math or the academic content of the report or the code. No Agentic AI tools has been used for wrriting the src code or the report. Agentic AI tools has been used for writing some of testing files in the tests directory or refactoring code as renaming files, variables or functions.

Indhold

1	Introduction	3
2	Software	4
2.1	Class overview	4
2.2	Design Principles	4
2.3	Primary Libraries	4
3	Theory	5
4	Results	6
5	Discussion	7
6	Conclusion	8
7	Future Work	9
8	Appendix	11
8.1	Source Code Repository	11
8.2	Source code & Envoriment	11
8.3	Class Diagrams	12
8.4	Class Design	14

1 Introduction

Hidden Markov Models (HMMs) are a powerful tool for modeling sequential data, where the underlying states of the system are not directly observable. They have been widely used in various fields such as speech recognition, natural language processing, and bioinformatics. [5].

The goal of this project is to inference if a person is in a room based on CO₂ data collected from a sensor in the room. To do this, we will use a Hidden Markov Model to model the relationship between the observed CO₂ levels and the hidden states of whether a person is in the room or not and if a window is open or not. When training the models, we will use **JAX** [2] and **EQUINOX** [4] for automatic differentiation and optimization, and we will use **Optax** [3] as our optimization library.

WRITE HERE THE RESULTS YOU FIND LATER ON.

This report is written for the purpose of given a concise outline of the projects achivements, design choices, model results and interpretaiton. The report is written for future students to be able to easily carry on the the project and extend it in some way.

2 Software

This section will explain the code design, libraries used and optimizers used. A repository link can be found in 8.1 and the environment details and short source code overview can be found in 8.2. Class diagrams can be found in 8.3 and code design choices will be explained in the following section.

2.1 Class overview

The source code is organized into 6 main components: the `HMM` class, the `Emissions` class, the `Transitions` class, the `HMMParams` class, the `Inference` class, and the `Solvers` class. All classes except `HMM` are abstract interfaces. The `HMM` class acts as the main entry point, providing a simple interface for training and inference by delegating to the other components. The key design principle is that concrete implementations of `Emissions` and `Transitions` are injected at runtime, allowing different HMM variants to be used interchangeably. A detailed explanation of each class and the motivation behind the design choices can be found in 8.4, and class diagrams are provided in 8.3.

2.2 Design Principles

A Object Orientated Programming (OOP) design was chosen for this project to promote modularity, extensibility, and separation of concerns.

The Design Inversion Principle (DIP) was a key principle in the design of the codebase, The use of abstract interfaces for the core components of the HMM allows for different implementations to be easily swapped in and out without affecting the rest of the system.

This design also promotes code reuse and makes it easier to extend the project in the future by adding new types of emissions, transitions, or inference algorithms without modifying existing code.

Overall, the OOP design provides a clear structure for the codebase and facilitates future development and maintenance of the project.

2.3 Primary Libraries

The primary libraries used in this projects are `JAX` [2], `EQUINOX` [4], and `Optax` [3].

`JAX` was chosen over `NumPy` for its automatic differentiation capabilities and performance optimizations. It allows for efficient computation of gradients, which is essential for training the HMM parameters using gradient-based optimization methods.

3 Theory

4 Results

5 Discussion

6 Conclusion

7 Future Work

Litteratur

- [1] Astral Software Inc. uv: An ultra-fast python package installer and resolver. <https://docs.astral.sh/uv/>, 2024. Accessed: 2026-05-30.
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [3] Matteo Hessel, David Budden, Fabio Viola, Mihaela Rosca, Eren Sezener, and Tom Hennigan. Optax: composable gradient transformation and optimisation, in jax!, 2020.
- [4] Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.
- [5] Ultralytics LLC. Hidden markov model (hmm) - real-world applications. <https://www.ultralytics.com/glossary/hidden-markov-model-hmm#real-world-applications>, 2024. Accessed: 2026-05-30.

8 Appendix

8.1 Source Code Repository

The repository is hosted on GitHub and the source code can be found there with the data as well: <https://github.com/Maccen26/hmm-project>

8.2 Source code & Envoriment

This code was written with Python (version 3.11) and `uv` (version 0.9.7) [1] was used as a package manager. The libraries used are listed in Table 1.

Library	Version
<code>dotenv</code>	$\geq 0.9.9$
<code>equinox</code>	$\geq 0.13.7$
<code>jax</code>	$\geq 0.9.0.1$
<code>matplotlib</code>	$\geq 3.10.8$
<code>numpy</code>	$\geq 1.23.5, < 2.3$
<code>optax</code>	$\geq 0.2.8$
<code>pandas</code>	$\geq 3.0.0$
<code>seaborn</code>	$\geq 0.13.2$
<code>statsmodels</code>	$\geq 0.14.6$

Tabel 1: Python libraries used in this project.

The source code is organized in a way such that the directory `src` contains the source code for the project, the directory `data` contains the data used for training and testing the model, the directory `tests` contains tests for the source code and the directory `drivers` contains scripts for running the model and generating the results used in the report.

8.3 Class Diagrams

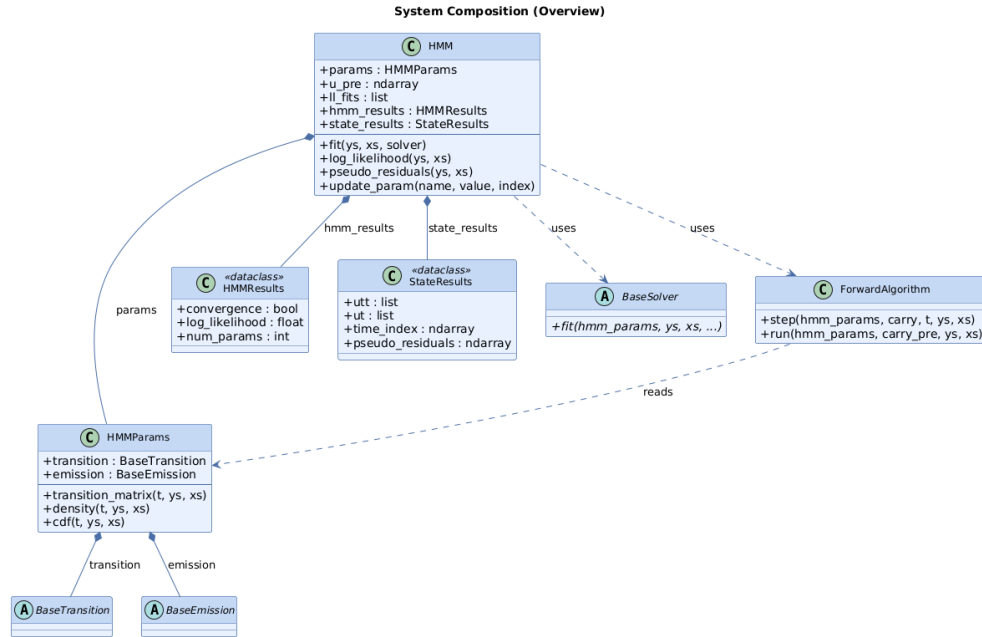


Figure 1: System composition overview. The user-facing `HMM` class owns an `HMMParams` instance composed of a `BaseTransition` and a `BaseEmission`. Fitting is delegated to a `BaseSolver` subclass and state inference is handled by `ForwardAlgorithm`, producing `HMMResults` and `StateResults`.

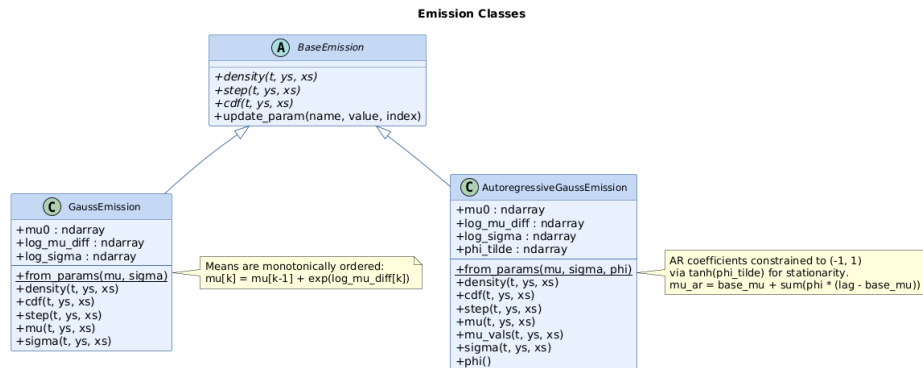


Figure 2: Emission class hierarchy. `GaussEmission` models state-dependent Gaussian distributions with monotonically ordered means enforced via cumulative exponentiation of `log_mu_diff`. `AutoregressiveGaussEmission` extends this with $AR(p)$ dynamics, where coefficients are stored unconstrained (`phi_tilde`) and mapped to $(-1, 1)$ via $\tanh$ to guarantee stationarity.

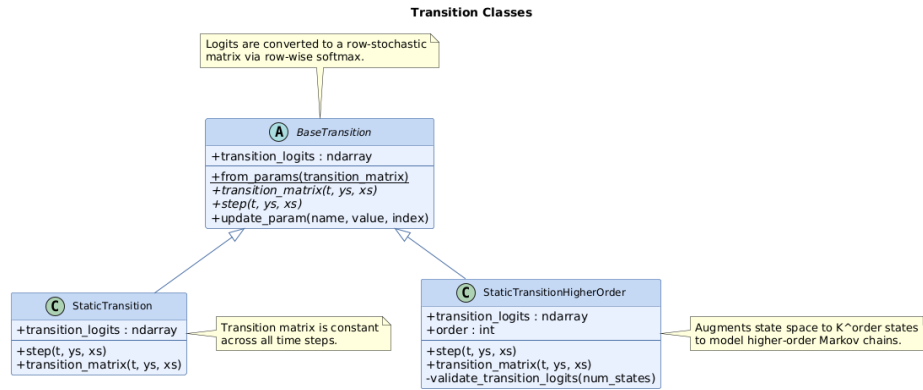


Figure 3: Transition class hierarchy. Both implementations store parameters as logits and convert them to a row-stochastic matrix via row-wise softmax. **StaticTransition** uses a constant $K \times K$ matrix, while **StaticTransitionHigherOrder** augments the state space to K^{order} states to represent a higher-order Markov chain.

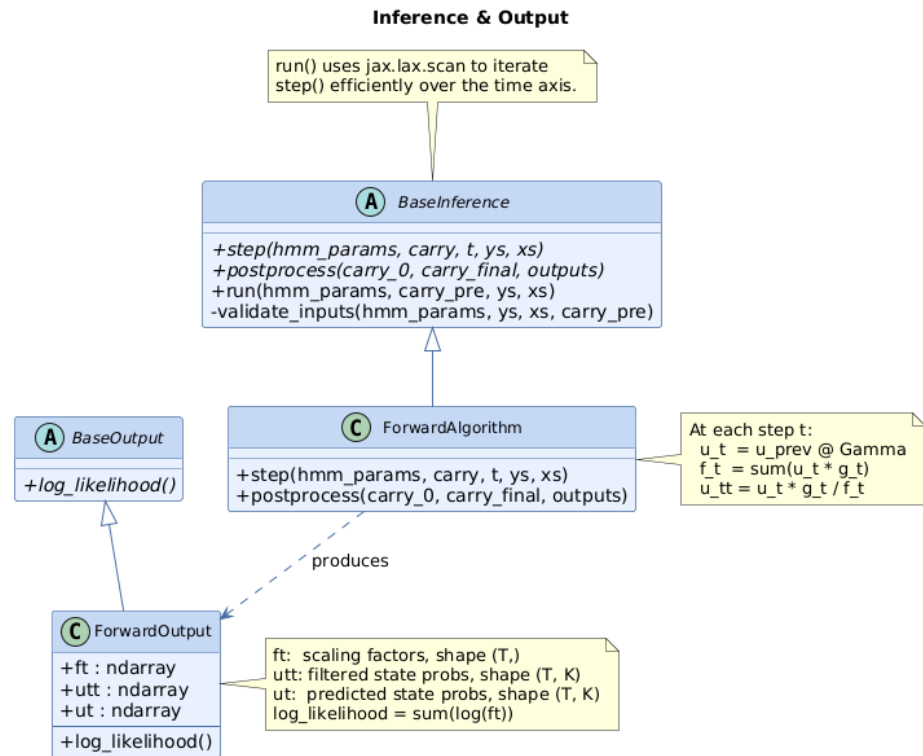


Figure 4: Inference and output class hierarchy. **BaseInference.run()** iterates **step()** across the time axis using `jax.lax.scan`. **ForwardAlgorithm** implements the forward filtering pass and produces a **ForwardOutput** storing the scaling factors f_t , filtered state probabilities $u_{t|t}$, and predicted state probabilities u_t . The log-likelihood is computed as $\sum_t \log f_t$.

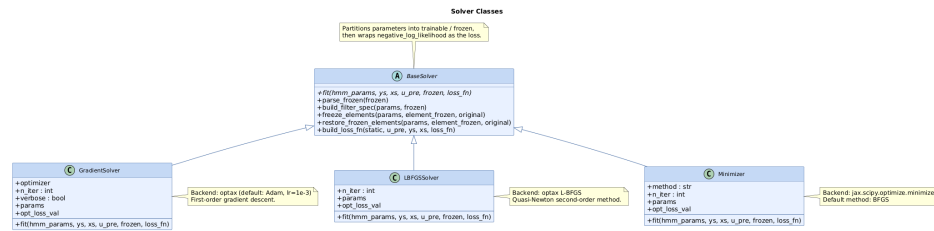


Figure 5: Solver class hierarchy. **BaseSolver** partitions parameters into trainable and frozen subsets and wraps the negative log-likelihood as the loss function. The three concrete solvers share the same interface but differ in their optimization backend: **GradientSolver** uses first-order gradient descent via **optax**, **LBFGSSolver** uses quasi-Newton L-BFGS via **optax**, and **Minimizer** wraps **jax.scipy.optimize.minimize**.

8.4 Class Design

The **HMM** class (1) is the main entry point of the system. Its responsibility is to act as a wrapper for the other components and to provide a simple interface for training and inference. It is dependent on **Emissions**, **Transitions**, **HMMParams**, and **Solvers**, which are composed together to form a complete HMM.

The **Emissions** (2) and **Transitions** (3) classes are abstract interfaces whose concrete implementations must be provided at runtime. This design choice is motivated by the structure of HMM variants: the differences between models in this project lie in how the density of an observation given a state is computed (emissions) and how the transition probabilities between states are computed (transitions). By separating these concerns into abstract interfaces, different HMM models can be composed without changing the rest of the system.

The **HMMParams** class acts as a container for the model parameters and as a wrapper for the methods implemented in the **Emissions** and **Transitions** classes. This is required by how **JAX** and **Equinox** work: parameters must be stored as pytrees to support automatic differentiation.

The **Inference** class (4) is an abstract interface responsible for implementing inference algorithms such as the forward algorithm, the backward algorithm, and the Viterbi algorithm. In this project, only the forward algorithm is implemented, but the design allows easy extension to other inference algorithms without modifying existing code.

The **Solvers** class (5) implements the optimization algorithms and depends on a loss function that takes the **HMMParams** and **Inference** classes as arguments. This dependency injection of the loss function decouples the optimizer from the objective, making it straightforward to change the loss function without altering the optimization algorithm.